

计算机网络(H) Lab-5 实验报告

姓名： 邱深凌

学号： 24300240123

Part 1

任务内容： 用traceroute发送大小为56 bytes的UDP包，比如|td>tracertofudan.edu.cn56
| |

运行环境 macOS 15.6.1

WireShark 4.6.1

校园网环境(iFudanNG.1x)

```

(base) vagrant@VagRantdeMacBook-Air ~ % traceroute fudan.edu.cn 56
traceroute to fudan.edu.cn (202.120.224.81), 64 hops max, 56 byte packets
 1  10.223.0.1 (10.223.0.1)  7.794 ms  4.288 ms  7.253 ms
 2  10.250.2.9 (10.250.2.9)  7.816 ms  10.235 ms  8.387 ms
 3  10.250.3.169 (10.250.3.169)  4225.796 ms  4008.900 ms  4646.861 ms
 4  * * *
 5  10.250.2.162 (10.250.2.162)  21.149 ms  11.844 ms  13.077 ms
 6  224.fudan.edu.cn (202.120.224.81)  12.014 ms  12.110 ms  11.242 ms
(base) vagrant@VagRantdeMacBook-Air ~ %
```

任务 1:第一个UDP包

1 / 24



10.223.131.202

****在IP header中，上层协议的数值是多少？****

Protocol UDP (17) 说明上层协议号是17

****IP header有多少bytes? IP数据报数据载荷有多少bytes? ****

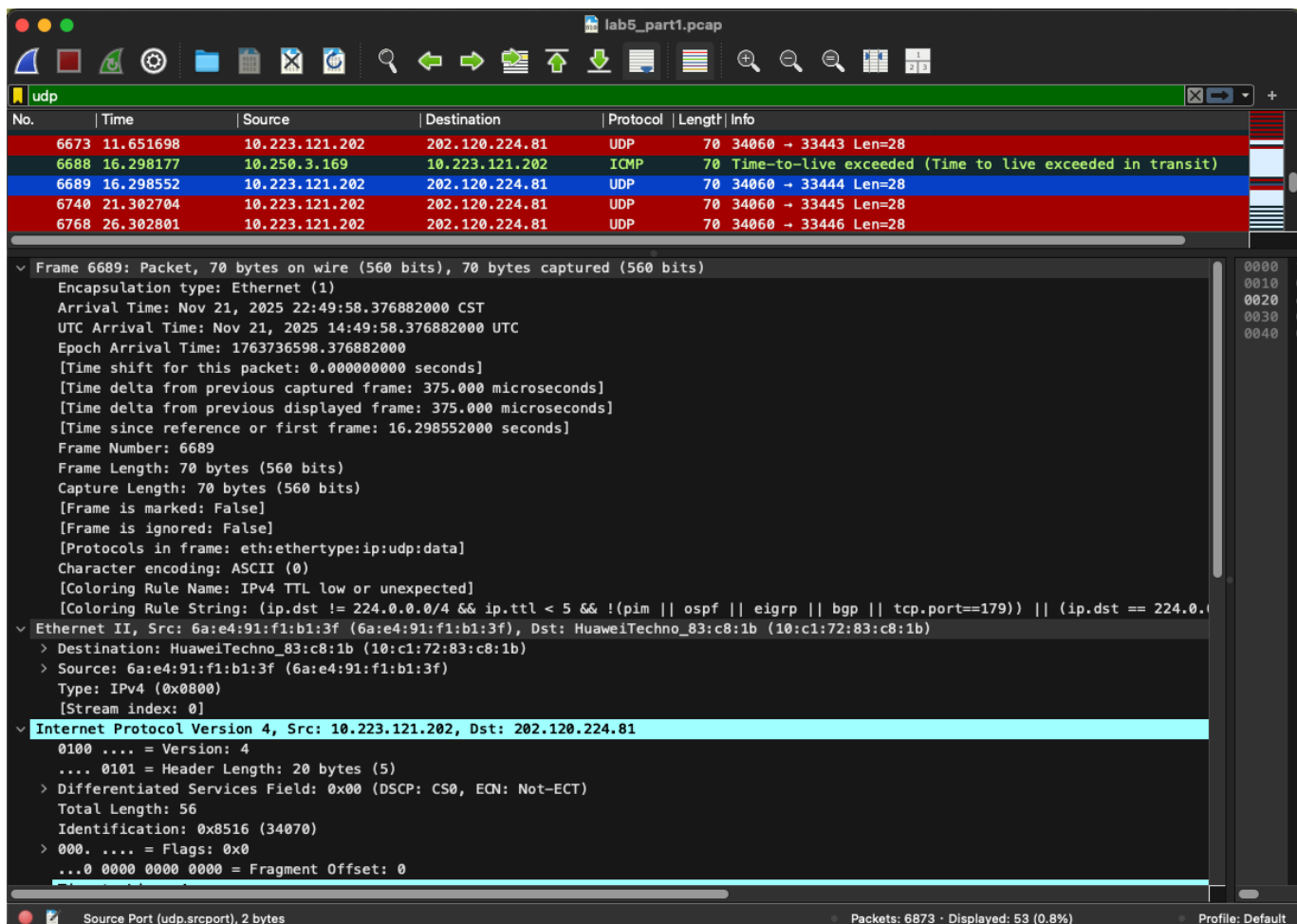
Header Length: 20 bytes (5) Total Length: 56, 所以IP 头部长度20 字节P 数据报的payload= 56 – 20 = 36 字节

在下面 UDP 部分还能看到 **UDP payload (28 bytes)**，因此UDP头8Bytes，数据28Bytes

该IP数据报是否分片？

Flags:0x0 说明未分片

任务 2:连续的UDP



Identification标识字段变化，Checksum变化（这都是显然的）

此外，UDP Dst变化，从33444变成33445；TTL也增加了

不变的字段即为剩下的字段，包括Header length / Total length / Src 和 Dst以及协议相关的字段例如分片等

为什么有些字段不断变化，为什么有些不变？

traceroute 的工作原理就是每一组探测包把 TTL 设置为 1、2、3、4...，让数据报在第 1、2、3、4 跳被丢弃从而得到每一跳的路由器信息。所以每次发送新的探测报文时TTL 字段必须改变

此外Identification和Checksum自然会变化，因为不是同一个包

UDP目的端口变化是因为traceroute 使用不同的 UDP 目的端口来区分每一个 probe，当收到 ICMP端口不可达时可以根据端口号对应到哪一次探测

其他则保持不变

列出连续IP数据报中的标识序列

0x8515 0x8516 0x8517 0x8518

任务 3:TTL-exceeded replies

The image shows two screenshots of the Wireshark network protocol analyzer. The top screenshot displays a list of captured packets, with packet 5803 selected. The bottom screenshot shows the detailed view of packet 5803, highlighting the ICMP Time-to-live exceeded message and the subsequent Internet Protocol Version 4 header.

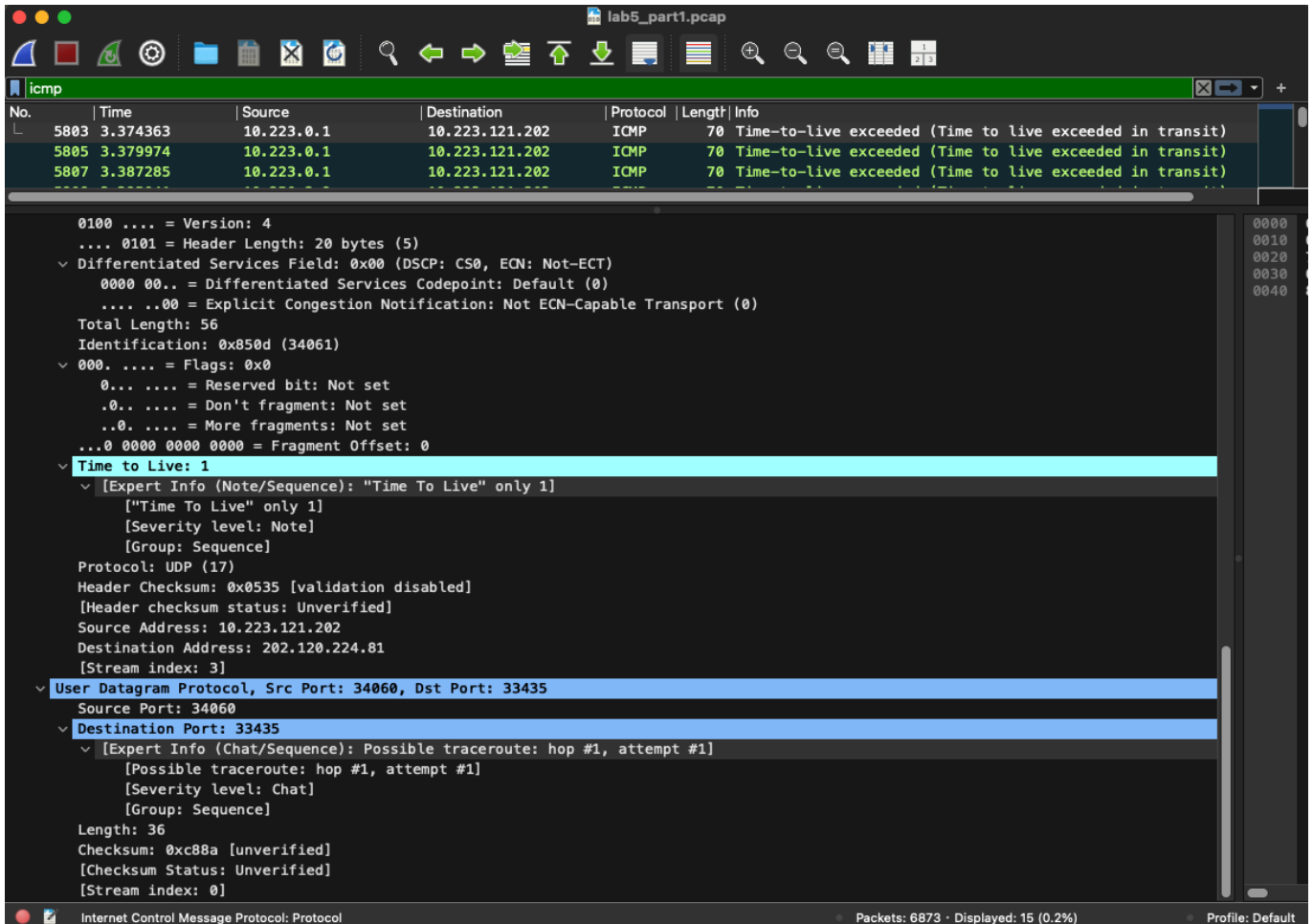
Packet List (Top Screenshot):

No.	Time	Source	Destination	Protocol	Length	Info
5803	3.374363	10.223.0.1	10.223.121.202	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
5805	3.379974	10.223.0.1	10.223.121.202	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
5807	3.387285	10.223.0.1	10.223.121.202	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)

Packet Details (Bottom Screenshot):

Frame 5803: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits)

- Encapsulation type: Ethernet (1)
- Arrival Time: Nov 21, 2025 22:49:45.452693000 CST
- UTC Arrival Time: Nov 21, 2025 14:49:45.452693000 UTC
- Epoch Arrival Time: 1763736585.452693000
- [Time shift for this packet: 0.000000000 seconds]
- [Time delta from previous captured frame: 6.942000 milliseconds]
- [Time since reference or first frame: 3.374363000 seconds]
- Frame Number: 5803
- Frame Length: 70 bytes (560 bits)
- Capture Length: 70 bytes (560 bits)
- [Frame is marked: False]
- [Frame is ignored: False]
- [Protocols in frame: eth:ethertype:ip:icmp:ip:udp]
- Character encoding: ASCII (0)
- [Coloring Rule Name: ICMP errors]
- [Coloring Rule String: icmp.type in { 3..5, 11 } || icmpv6.type in { 1..4 }]
- Ethernet II, Src: HuaweiTechno_83:c8:1b (10:c1:72:83:c8:1b), Dst: 6a:e4:91:f1:b1:3f (6a:e4:91:f1:b1:3f)**
 - Destination: 6a:e4:91:f1:b1:3f (6a:e4:91:f1:b1:3f)
 - ... 1. = LG bit: Locally administered address (this is NOT the factory default)
 - ... 0. = IG bit: Individual address (unicast)
 - Source: HuaweiTechno_83:c8:1b (10:c1:72:83:c8:1b)
 - ... 0. = LG bit: Globally unique address (factory default)
 - ... 0. = IG bit: Individual address (unicast)
 - Type: IPv4 (0x0800)
 - [Stream index: 0]
- Internet Protocol Version 4, Src: 10.223.0.1, Dst: 10.223.121.202**
 - 0100 = Version: 4
 - 0101 = Header Length: 20 bytes (5)
 - Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)
 - 1100 00.. = Differentiated Services Codepoint: Class Selector 6 (48)
 -00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
 - Total Length: 56
 - Identification: 0x02c7 (711)
 - 000. = Flags: 0x0
 - 0... = Reserved bit: Not set
 - .0.. = Don't fragment: Not set
 - ..0. = More fragments: Not set
 - ...0 0000 0000 0000 = Fragment Offset: 0
 - Time to Live: 255
 - Protocol: ICMP (1)
 - Header Checksum: 0x28b5 [validation disabled]
 - [Header checksum status: Unverified]
 - Source Address: 10.223.0.1
 - Destination Address: 10.223.121.202
 - [Stream index: 4]
- Internet Control Message Protocol**
 - Type: Time-to-live exceeded (11)**
 - [Expert Info (Note/Response): Type indicates an error]
 - [Type indicates an error]
 - [Severity level: Note]
 - [Group: Response]
 - Code: 0 (Time to live exceeded in transit)
 - Checksum: 0x24a9 [correct]
 - [Checksum Status: Good]
 - Unused: 00000000
 - Internet Protocol Version 4, Src: 10.223.121.202, Dst: 202.120.224.81**
 - 0100 = Version: 4
 - 0101 = Header Length: 20 bytes (5)
 - Differentiated Services Field: 0xc0 (DSCP: CS6, ECN: Not-ECT)



标识字段与TTL字段分别是多少？

Identification: 0x02c7 (711) Time to Live: 255

即标识字段 (Identification) = 0x02c7 (711), TTL = 255

收到的所有TTL-exceeded replies中, 这两个字段是否不变? 为什么?

收到的 TTL-exceeded 报文中, Identification 字段随每个报文改变, 因为报文不是一个, 标识不同很合理

而 TTL 字段基本保持不变 (实际上观察到255 255 255 254 254 253...), 因为路由器为发出的 ICMP 报文设置了相同的初始 TTL, 并且到达源主机的路径比较接近 (主要因为校园网到fudan.edu.cn很近, 所以路径都接近) 所以TTL几乎相同

Part 2

任务 1: 定制化拓扑

代码说明

开发环境: windows VMWare WorkStation

系统: Ubuntu 24.04LTS Desktop

mininet 2.3.0 iperf 2.1.9

首先定义了一个继承 Topo 的自定义拓扑类 **MyTopo**，之后一共添加了 4 台主机 H1 H2 H3 H4 并手动指定了 IP 地址和 MAC 地址，之后定义两个交换机 s1 s2（其实就是按照图示结构），并定义了链路

```
self.addLink(h1, s1, bw=10, delay="2ms")
```

检验和分析

通过 `sudo mn --custom ./customized_topo.py --topo mytopo --test pingall --link tc` 指令检验，并截图。描述一下出现的现象，并阐述一下原因

运行的完整截图：

```

Nov 21 12:35
computerNetworkLab5
customized_topo.py
1 from mininet.topo import Topo
2 from mininet.net import Mininet
3 from mininet.node import OVSController
4 from mininet.link import TCLink
5 from mininet.log import setLogLevel
6
7
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo mn -c
rm -f ~/.ssh/mn/*
*** Cleanup complete.
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo mn --custom ./customized_topo.py --topo mytopo --test pingall --link tc
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
H1 H2 H3 H4
*** Adding switches:
S1 S2
*** Adding links:
(10.00Mbit 2ms delay) (10.00Mbit 2ms delay) (H1, S1) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(H2, S1) (10.00Mbit 2ms delay) (10.00Mbit 2ms delay) (H3, S2) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(H4, S2) (20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(S1, S2)
*** Configuring hosts
H1 H2 H3 H4
*** Starting controller

```

```

Nov 21 12:36
computerNetworkLab5
customized_topo.py
1 from mininet.topo import Topo
2 from mininet.net import Mininet
3 from mininet.node import OVSController
4 from mininet.link import TCLink
5
6
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo mn --custom ./customized_topo.py --topo mytopo --test pingall --link tc
H1 H2 H3 H4
*** Starting controller

*** Starting 2 switches
S1 S2 ... (10.00Mbit 2ms delay) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(10.00Mbit 2ms delay) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.

*** Waiting for switches to connect
S1 S2
*** Ping: testing ping reachability
H1 -> H2 X H4
H2 -> H1 H3 H4
H3 -> H1 H2 H4
H4 -> X H2 H3
*** Results: 16% dropped (10/12 received)
*** Stopping 0 controllers

*** Stopping 5 links
....
*** Stopping 2 switches
S1 S2
*** Stopping 4 hosts
H1 H2 H3 H4
*** Done
completed in 22.913 seconds
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$

```

可以看到16%dropped (10/12)

为了保证严谨，我多进行了几次实验，25%dropped (9/12)

```

Nov 21 13:01
File Edit Selection View Go ...
computerNetworkLab5
EXPLORER
customized_topo.py
COMPUTERNETWORKLABS
customized_topo.py
1 from mininet.topo import Topo
2 from mininet.net import Mininet
3 from mininet.node import OVSController
4 from mininet.link import TCLink

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(bash)
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo mn --custom ./customized_topo.py --topo
o mytopo --test pingall --link tc
H1 H2 H3 H4
*** Starting controller

*** Starting 2 switches
S1 S2 ... (10.00Mbit 2ms delay) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider
r2q change.
(20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(10.00Mbit 2ms delay) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q chang
e.
(20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.

*** Waiting for switches to connect
S1 S2
*** Ping: testing ping reachability
H1 -> H2 H3 H4
H2 -> H1 H3 H4
H3 -> X H2 H4
H4 -> X H2 H3
*** Results: 25% dropped (9/12 received)
*** Stopping 0 controllers

*** Stopping 5 links
....
*** Stopping 2 switches
S1 S2
*** Stopping 4 hosts
H1 H2 H3 H4
*** Done
completed in 24.811 seconds
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$
Ln 8, Col 20 Spaces: 4 UTF-8 LF {} Python 3.12.3

```

有丢包率低的, 8%dropped(11/12)

```

Nov 21 13:01
File Edit Selection View Go ...
computerNetworkLab5
EXPLORER
customized_topo.py
COMPUTERNETWORKLABS
customized_topo.py
1 from mininet.topo import Topo
2 from mininet.net import Mininet
3 from mininet.node import OVSController
4 from mininet.link import TCLink

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(bash)
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo mn --custom ./customized_topo.py --topo
o mytopo --test pingall --link tc
H1 H2 H3 H4
*** Starting controller

*** Starting 2 switches
S1 S2 ... (10.00Mbit 2ms delay) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider
r2q change.
(20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(10.00Mbit 2ms delay) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q chang
e.
(20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.

*** Waiting for switches to connect
S1 S2
*** Ping: testing ping reachability
H1 -> H2 H3 H4
H2 -> H1 H3 H4
H3 -> H1 H2 H4
H4 -> X H2 H3
*** Results: 8% dropped (11/12 received)
*** Stopping 0 controllers

*** Stopping 5 links
....
*** Stopping 2 switches
S1 S2
*** Stopping 4 hosts
H1 H2 H3 H4
*** Done
completed in 12.996 seconds
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$
Ln 8, Col 20 Spaces: 4 UTF-8 LF {} Python 3.12.3

```

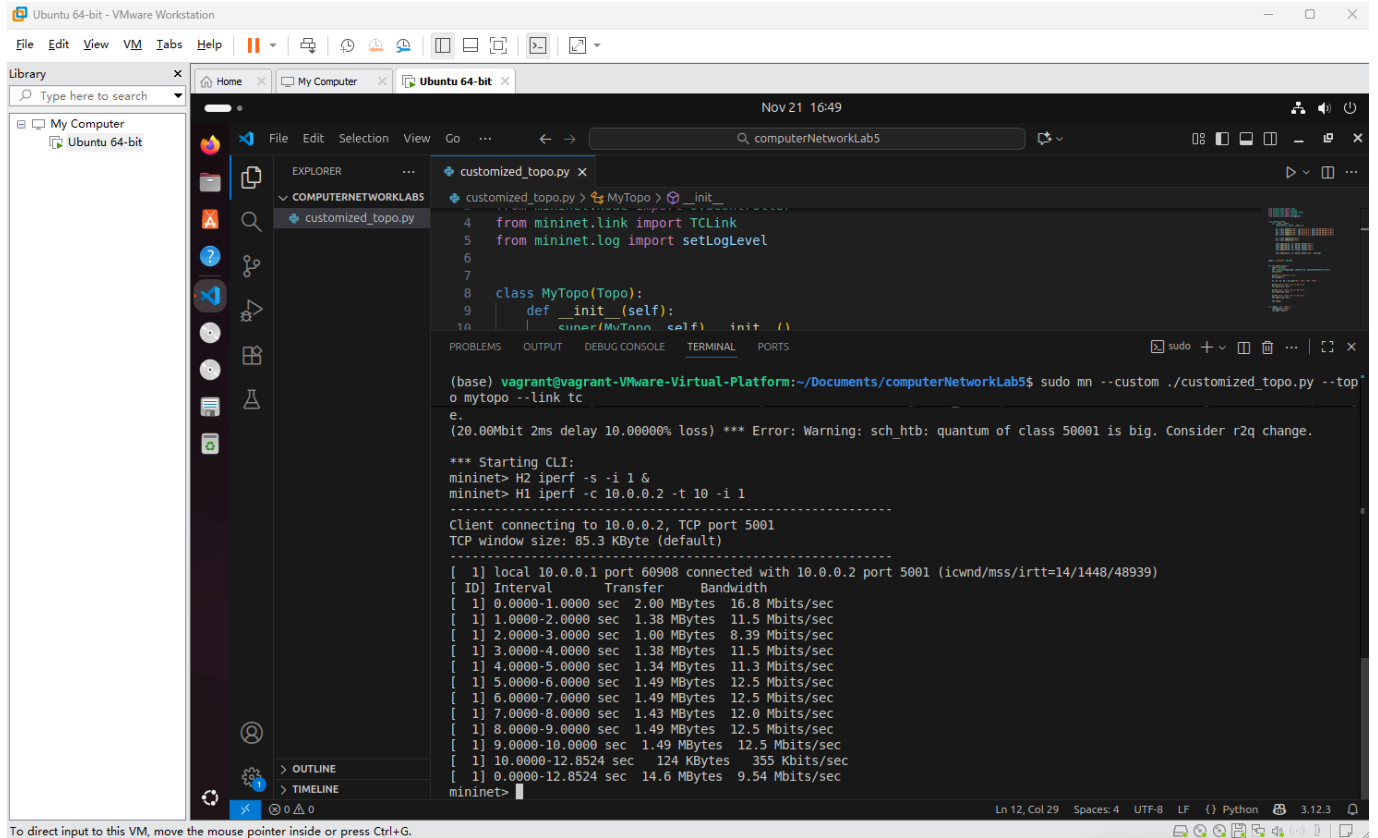
甚至还有没有丢包的, 0%dropped(12/12)

不过经过多组实验, 丢包率的平均值大概还是8% 16%即丢失一两个包, 也就是10%附近

2. iperf 分析 因为环境的冲突，直接**iperf H1 H2**会出现卡住的情况；因为这个冲突非常难解决，所以直接使用：

```
mininet> H4 iperf -s -i 1 &
mininet> H2 iperf -c 10.0.0.4 -t 10 -i 1
```

这样的方式去使用iperf，得到以下三组结果：

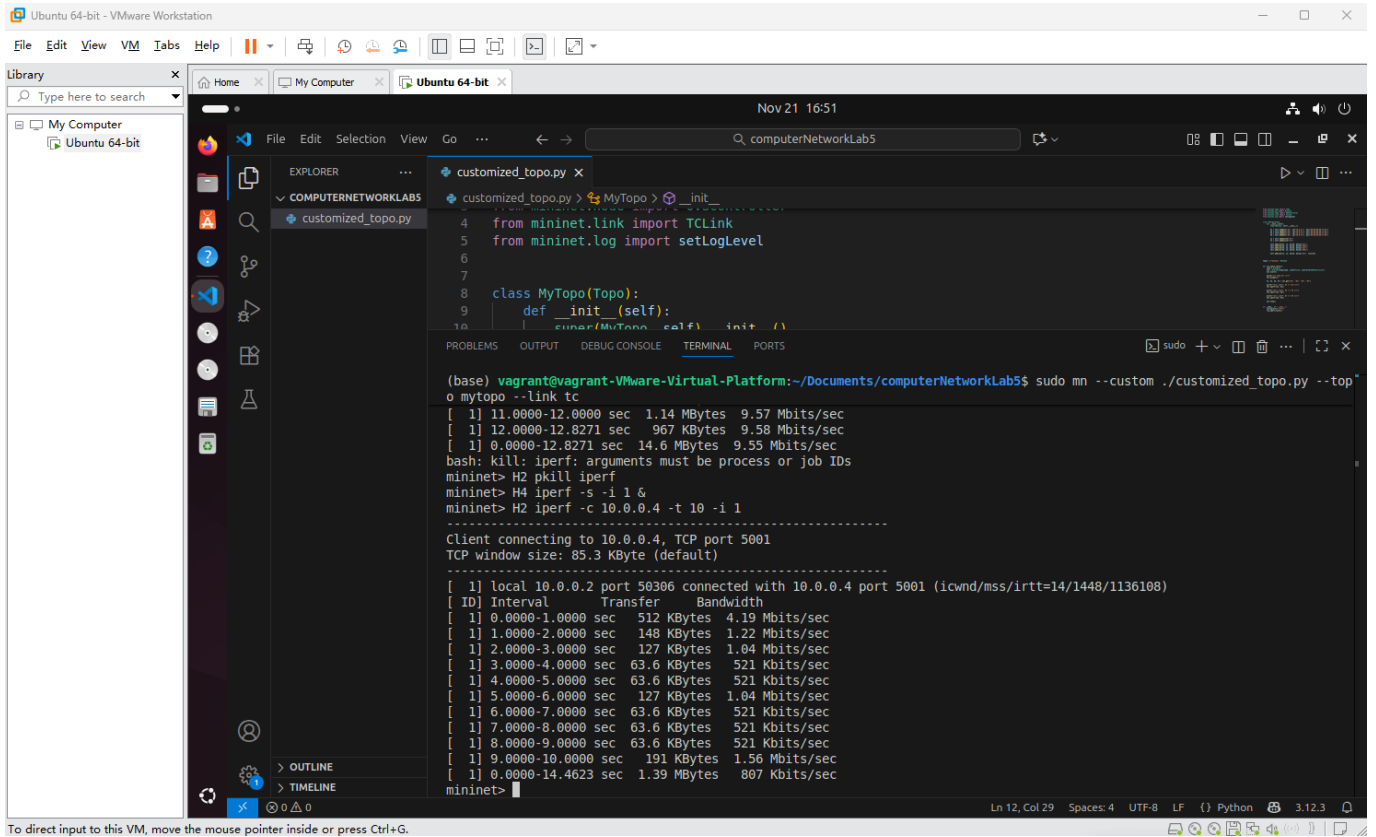


```

(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo mn --custom ./customized_topo.py --topo mytopo --link tc
e.
(20.00Mbit 2ms delay 10.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.

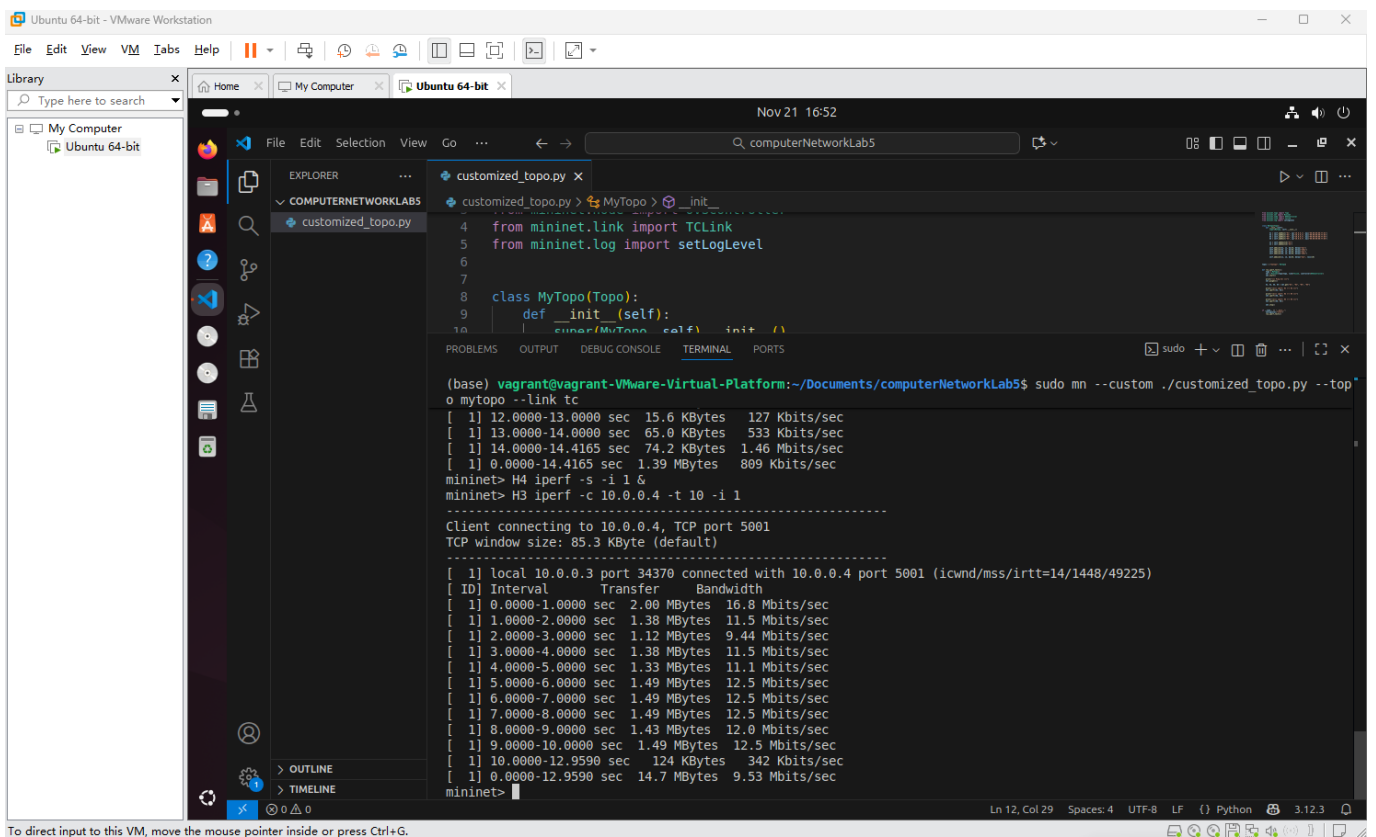
*** Starting CLI:
mininet> H2 iperf -s -i 1 &
mininet> H1 iperf -c 10.0.0.2 -t 10 -i 1
-----
Client connecting to 10.0.0.2, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 60908 connected with 10.0.0.2 port 5001 (icwnd/mss/irrt=14/1448/48939)
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-1.0000 sec  2.00 MBytes  16.8 Mbits/sec
[ 1] 1.0000-2.0000 sec  1.38 MBytes  11.5 Mbits/sec
[ 1] 2.0000-3.0000 sec  1.00 MBytes  8.39 Mbits/sec
[ 1] 3.0000-4.0000 sec  1.38 MBytes  11.5 Mbits/sec
[ 1] 4.0000-5.0000 sec  1.34 MBytes  11.3 Mbits/sec
[ 1] 5.0000-6.0000 sec  1.49 MBytes  12.5 Mbits/sec
[ 1] 6.0000-7.0000 sec  1.49 MBytes  12.5 Mbits/sec
[ 1] 7.0000-8.0000 sec  1.43 MBytes  12.0 Mbits/sec
[ 1] 8.0000-9.0000 sec  1.49 MBytes  12.5 Mbits/sec
[ 1] 9.0000-10.0000 sec 1.49 MBytes  12.5 Mbits/sec
[ 1] 10.0000-12.8524 sec 124 KBytes   355 Kbits/sec
[ 1] 0.0000-12.8524 sec 14.6 MBytes  9.54 Mbits/sec
mininet>
```

现象：H1 H2之间的吞吐量略有震荡，从8-16Mb/s都有，但是基本还是稳定在12Mb/s的（最后355的是异常值不管了），整体平均是9.54Mb/s，也是符合瓶颈链路的。



```
Nov 21 16:51
File Edit Selection View Go ...
customized_topo.py x
customized_topo.py > MyTopo > _init_
4 from mininet.link import TcLink
5 from mininet.log import setLogLevel
6
7
8 class MyTopo(Topo):
9     def _init_(self):
10         super(MyTopo, self). _init_ ()
11
12 (base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo mn --custom ./customized_topo.py --topo
o mytopo --link tc
[ 1] 11.0000-12.0000 sec 1.14 MBytes 9.57 Mb/s/sec
[ 1] 12.0000-12.8271 sec 967 KBytes 9.58 Mb/s/sec
[ 1] 0.0000-12.8271 sec 14.6 MBytes 9.55 Mb/s/sec
bash: kill: iperf: arguments must be process or job IDs
mininet> H2 iperf -s -i 1 &
mininet> H1 iperf -c 10.0.0.4 -t 10 -i 1
-----
Client connecting to 10.0.0.4, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 50306 connected with 10.0.0.4 port 5001 (icwnd/mss/irtt=14/1448/1136108)
[ ID] Interval Transfer Bandwidth
[ 1] 0.0000-1.0000 sec 512 KBytes 4.19 Mb/s/sec
[ 1] 1.0000-2.0000 sec 148 KBytes 1.22 Mb/s/sec
[ 1] 2.0000-3.0000 sec 127 KBytes 1.04 Mb/s/sec
[ 1] 3.0000-4.0000 sec 63.6 KBytes 521 Kb/s/sec
[ 1] 4.0000-5.0000 sec 63.6 KBytes 521 Kb/s/sec
[ 1] 5.0000-6.0000 sec 127 KBytes 1.04 Mb/s/sec
[ 1] 6.0000-7.0000 sec 63.6 KBytes 521 Kb/s/sec
[ 1] 7.0000-8.0000 sec 63.6 KBytes 521 Kb/s/sec
[ 1] 8.0000-9.0000 sec 63.6 KBytes 521 Kb/s/sec
[ 1] 9.0000-10.0000 sec 191 KBytes 1.56 Mb/s/sec
[ 1] 0.0000-14.4623 sec 1.39 MBytes 807 Kb/s/sec
mininet>
```

现象：H2 H4之间的吞吐量就非常不稳定，有时有1.几Mb/s但是基本都是几百Kb/s，证明10%的丢包率影响很大（也和任务二最后的结论相同），因此平均值807Kb/s，远远低于20Mb/s。证明TCP对于丢包很敏感



```
Nov 21 16:52
File Edit Selection View Go ...
customized_topo.py x
customized_topo.py > MyTopo > _init_
4 from mininet.link import TcLink
5 from mininet.log import setLogLevel
6
7
8 class MyTopo(Topo):
9     def _init_(self):
10         super(MyTopo, self). _init_ ()
11
12 (base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo mn --custom ./customized_topo.py --topo
o mytopo --link tc
[ 1] 12.0000-13.0000 sec 15.6 KBytes 127 Kb/s/sec
[ 1] 13.0000-14.0000 sec 65.0 KBytes 533 Kb/s/sec
[ 1] 14.0000-14.4165 sec 74.2 KBytes 1.46 Mb/s/sec
[ 1] 0.0000-14.4165 sec 1.39 MBytes 809 Kb/s/sec
mininet> H4 iperf -s -i 1 &
mininet> H3 iperf -c 10.0.0.4 -t 10 -i 1
-----
Client connecting to 10.0.0.4, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 34370 connected with 10.0.0.4 port 5001 (icwnd/mss/irtt=14/1448/49225)
[ ID] Interval Transfer Bandwidth
[ 1] 0.0000-1.0000 sec 2.00 MBytes 16.8 Mb/s/sec
[ 1] 1.0000-2.0000 sec 1.38 MBytes 11.5 Mb/s/sec
[ 1] 2.0000-3.0000 sec 1.12 MBytes 9.44 Mb/s/sec
[ 1] 3.0000-4.0000 sec 1.38 MBytes 11.5 Mb/s/sec
[ 1] 4.0000-5.0000 sec 1.33 MBytes 11.1 Mb/s/sec
[ 1] 5.0000-6.0000 sec 1.49 MBytes 12.5 Mb/s/sec
[ 1] 6.0000-7.0000 sec 1.49 MBytes 12.5 Mb/s/sec
[ 1] 7.0000-8.0000 sec 1.49 MBytes 12.5 Mb/s/sec
[ 1] 8.0000-9.0000 sec 1.43 MBytes 12.0 Mb/s/sec
[ 1] 9.0000-10.0000 sec 1.49 MBytes 12.5 Mb/s/sec
[ 1] 10.0000-12.9590 sec 124 KBytes 342 Kb/s/sec
[ 1] 0.0000-12.9590 sec 14.7 MBytes 9.53 Mb/s/sec
mininet>
```

现象：H3 H4的吞吐量和H1 H2的情况非常接近，平均是9.53Mb/s，也符合瓶颈链路

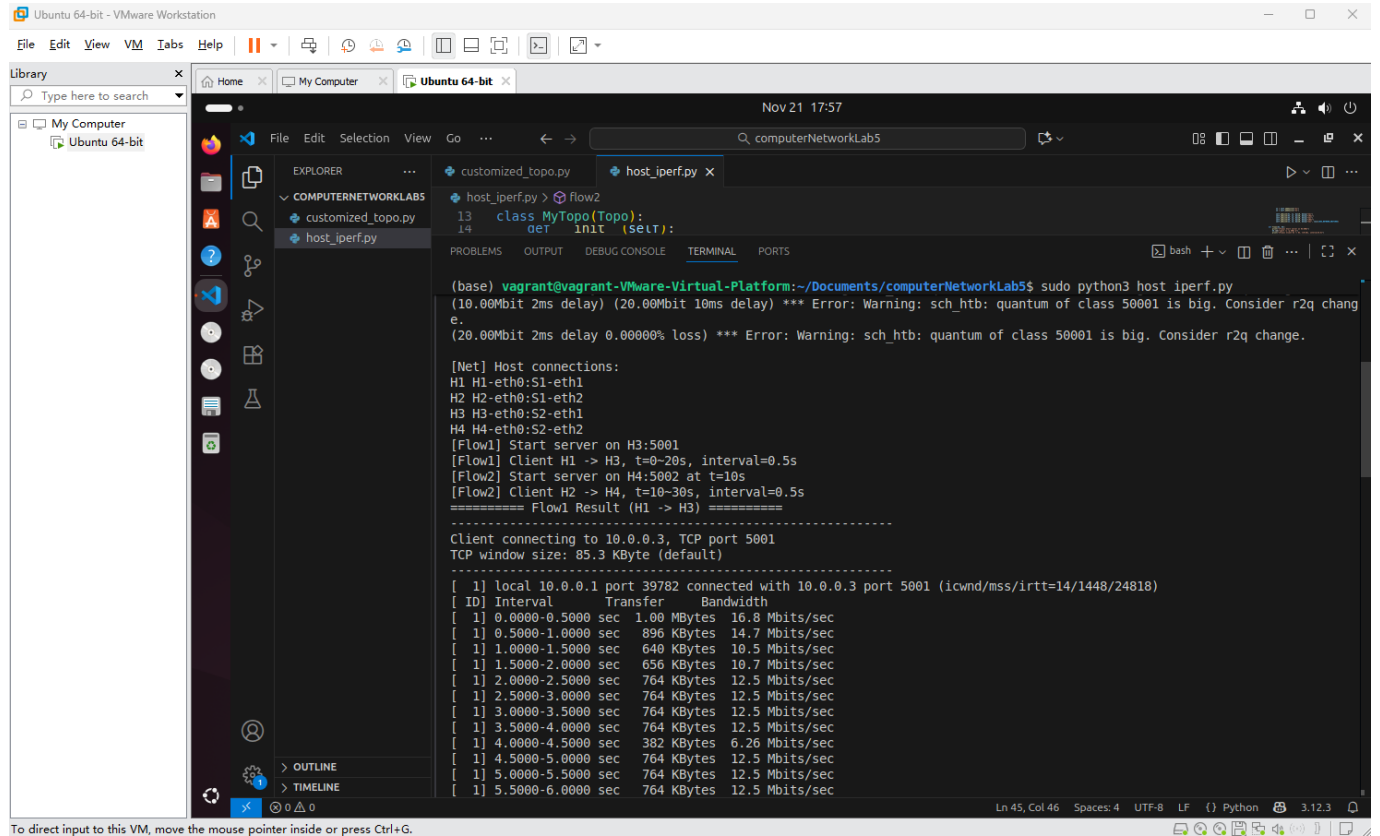
任务 2

代码说明

定义了一个全局变量 `LOSS_BETWEEN_SWITCHES`，表示交换机之间链路丢包率（用作比较），其他定义和任务1相同。对于TCP流的定义，`flow1`是H1到H3，从0s开始持续20s(-t 20)，每0.5s打印结果；`flow2`是从H2到H4，从10s开始（通过`time.sleep(10)`实现）

之后通过thread并行启动两个流

现象观察与分析



```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host_iperf.py
(10.00Mbit 2ms delay) (20.00Mbit 10ms delay) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.
(20.00Mbit 2ms delay 0.00000% loss) *** Error: Warning: sch_htb: quantum of class 50001 is big. Consider r2q change.

[Net] Host connections:
H1 H1-eth0:S1-eth1
H2 H2-eth0:S1-eth2
H3 H3-eth0:S2-eth1
H4 H4-eth0:S2-eth2
[Flow1] Start server on H3:5001
[Flow1] Client H1 -> H3, t=0-20s, interval=0.5s
[Flow2] Start server on H4:5002 at t=10s
[Flow2] Client H2 -> H4, t=10-30s, interval=0.5s
===== Flow1 Result (H1 -> H3) =====
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
[ 1] local 10.0.0.1 port 39782 connected with 10.0.0.3 port 5001 (icwnd/mss/irtt=14/1448/24818)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-0.5000 sec  1.00 MBytes  16.8 Mbits/sec
[ 1] 0.5000-1.0000 sec   896 KBytes  14.7 Mbits/sec
[ 1] 1.0000-1.5000 sec   640 KBytes  10.5 Mbits/sec
[ 1] 1.5000-2.0000 sec   656 KBytes  10.7 Mbits/sec
[ 1] 2.0000-2.5000 sec   764 KBytes  12.5 Mbits/sec
[ 1] 2.5000-3.0000 sec   764 KBytes  12.5 Mbits/sec
[ 1] 3.0000-3.5000 sec   764 KBytes  12.5 Mbits/sec
[ 1] 3.5000-4.0000 sec   764 KBytes  12.5 Mbits/sec
[ 1] 4.0000-4.5000 sec   382 KBytes   6.26 Mbits/sec
[ 1] 4.5000-5.0000 sec   764 KBytes  12.5 Mbits/sec
[ 1] 5.0000-5.5000 sec   764 KBytes  12.5 Mbits/sec
[ 1] 5.5000-6.0000 sec   764 KBytes  12.5 Mbits/sec
```

一共进行了4次实验，分别设置丢包率为0% 1% 5% 10%

丢包率为0%

===== Flow1 Result (H1 -> H3) =====

Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)

[1] local 10.0.0.1 port 39782 connected with 10.0.0.3 port 5001 (icwnd/mss/irrt=14/1448/24818)

[ID]	Interval	Transfer	Bandwidth
[1]	0.0000-0.5000 sec	1.00 MBytes	16.8 Mbits/sec
[1]	0.5000-1.0000 sec	896 KBytes	14.7 Mbits/sec
[1]	1.0000-1.5000 sec	640 KBytes	10.5 Mbits/sec
[1]	1.5000-2.0000 sec	656 KBytes	10.7 Mbits/sec
[1]	2.0000-2.5000 sec	764 KBytes	12.5 Mbits/sec
[1]	2.5000-3.0000 sec	764 KBytes	12.5 Mbits/sec
[1]	3.0000-3.5000 sec	764 KBytes	12.5 Mbits/sec
[1]	3.5000-4.0000 sec	764 KBytes	12.5 Mbits/sec
[1]	4.0000-4.5000 sec	382 KBytes	6.26 Mbits/sec
[1]	4.5000-5.0000 sec	764 KBytes	12.5 Mbits/sec
[1]	5.0000-5.5000 sec	764 KBytes	12.5 Mbits/sec
[1]	5.5000-6.0000 sec	764 KBytes	12.5 Mbits/sec
[1]	6.0000-6.5000 sec	764 KBytes	12.5 Mbits/sec
[1]	6.5000-7.0000 sec	764 KBytes	12.5 Mbits/sec
[1]	7.0000-7.5000 sec	764 KBytes	12.5 Mbits/sec
[1]	7.5000-8.0000 sec	764 KBytes	12.5 Mbits/sec
[1]	8.0000-8.5000 sec	764 KBytes	12.5 Mbits/sec
[1]	8.5000-9.0000 sec	764 KBytes	12.5 Mbits/sec
[1]	9.0000-9.5000 sec	764 KBytes	12.5 Mbits/sec
[1]	9.5000-10.0000 sec	764 KBytes	12.5 Mbits/sec
[1]	10.0000-10.5000 sec	764 KBytes	12.5 Mbits/sec
[1]	10.5000-11.0000 sec	636 KBytes	10.4 Mbits/sec
[1]	11.0000-11.5000 sec	1019 KBytes	16.7 Mbits/sec
[1]	11.5000-12.0000 sec	1.12 MBytes	18.8 Mbits/sec
[1]	12.0000-12.5000 sec	1.06 MBytes	17.7 Mbits/sec
[1]	12.5000-13.0000 sec	1.18 MBytes	19.8 Mbits/sec
[1]	13.0000-13.5000 sec	1.18 MBytes	19.8 Mbits/sec
[1]	13.5000-14.0000 sec	1.18 MBytes	19.8 Mbits/sec
[1]	14.0000-14.5000 sec	1.43 MBytes	24.0 Mbits/sec
[1]	14.5000-15.0000 sec	572 KBytes	9.37 Mbits/sec

(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5\$ sudo python3 host iperf.py

[1]	14.0000-14.5000 sec	1.43 MBytes	24.0 Mbits/sec
[1]	14.5000-15.0000 sec	572 KBytes	9.37 Mbits/sec
[1]	15.0000-15.5000 sec	1.43 MBytes	24.0 Mbits/sec
[1]	15.5000-16.0000 sec	1.30 MBytes	21.9 Mbits/sec
[1]	16.0000-16.5000 sec	1.37 MBytes	22.9 Mbits/sec
[1]	16.5000-17.0000 sec	1.68 MBytes	28.2 Mbits/sec
[1]	17.0000-17.5000 sec	1.62 MBytes	27.1 Mbits/sec
[1]	17.5000-18.0000 sec	1.55 MBytes	26.1 Mbits/sec
[1]	18.0000-18.5000 sec	1.37 MBytes	22.9 Mbits/sec
[1]	18.5000-19.0000 sec	507 KBytes	8.30 Mbits/sec
[1]	19.0000-19.5000 sec	637 KBytes	10.4 Mbits/sec
[1]	19.5000-20.0000 sec	381 KBytes	6.24 Mbits/sec
[1]	20.0000-33.3505 sec	127 KBytes	77.6 Kbits/sec
[1]	0.0000-33.3505 sec	36.7 MBytes	9.23 Mbits/sec

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
[ 1] 20.0000-33.3505 sec 127 KBytes 77.6 Kbits/sec
[ 1] 0.0000-33.3505 sec 36.7 MBytes 9.23 Mbits/sec

===== Flow2 Result (H2 -> H4) =====
-----
Client connecting to 10.0.0.4, TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 36362 connected with 10.0.0.4 port 5002 (icwnd/mss/irrt=14/1448/89488)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-0.5000 sec 1.50 MBytes 25.2 Mbits/sec
[ 1] 0.5000-1.0000 sec 939 KBytes 15.4 Mbits/sec
[ 1] 1.0000-1.5000 sec 763 KBytes 12.5 Mbits/sec
[ 1] 1.5000-2.0000 sec 827 KBytes 13.6 Mbits/sec
[ 1] 2.0000-2.5000 sec 764 KBytes 12.5 Mbits/sec
[ 1] 2.5000-3.0000 sec 827 KBytes 13.6 Mbits/sec
[ 1] 3.0000-3.5000 sec 892 KBytes 14.6 Mbits/sec
[ 1] 3.5000-4.0000 sec 699 KBytes 11.5 Mbits/sec
[ 1] 4.0000-4.5000 sec 827 KBytes 13.6 Mbits/sec
[ 1] 4.5000-5.0000 sec 892 KBytes 14.6 Mbits/sec
[ 1] 5.0000-5.5000 sec 699 KBytes 11.5 Mbits/sec
[ 1] 5.5000-6.0000 sec 892 KBytes 14.6 Mbits/sec
[ 1] 6.0000-6.5000 sec 699 KBytes 11.5 Mbits/sec
[ 1] 6.5000-7.0000 sec 827 KBytes 13.6 Mbits/sec
[ 1] 7.0000-7.5000 sec 764 KBytes 12.5 Mbits/sec
[ 1] 7.5000-8.0000 sec 827 KBytes 13.6 Mbits/sec
[ 1] 8.0000-8.5000 sec 892 KBytes 14.6 Mbits/sec
[ 1] 8.5000-9.0000 sec 699 KBytes 11.5 Mbits/sec
[ 1] 9.0000-9.5000 sec 827 KBytes 13.6 Mbits/sec
[ 1] 9.5000-10.0000 sec 764 KBytes 12.5 Mbits/sec
[ 1] 10.0000-10.5000 sec 510 KBytes 8.35 Mbits/sec
[ 1] 10.5000-11.0000 sec 699 KBytes 11.5 Mbits/sec
[ 1] 11.0000-11.5000 sec 955 KBytes 15.7 Mbits/sec
[ 1] 11.5000-12.0000 sec 1019 KBytes 16.7 Mbits/sec
[ 1] 12.0000-12.5000 sec 1.06 MBytes 17.7 Mbits/sec
[ 1] 12.5000-13.0000 sec 1.18 MBytes 19.8 Mbits/sec
[ 1] 13.0000-13.5000 sec 1.24 MBytes 20.9 Mbits/sec
```

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
[ 1] 8.0000-8.5000 sec 892 KBytes 14.6 Mbits/sec
[ 1] 8.5000-9.0000 sec 699 KBytes 11.5 Mbits/sec
[ 1] 9.0000-9.5000 sec 827 KBytes 13.6 Mbits/sec
[ 1] 9.5000-10.0000 sec 764 KBytes 12.5 Mbits/sec
[ 1] 10.0000-10.5000 sec 510 KBytes 8.35 Mbits/sec
[ 1] 10.5000-11.0000 sec 699 KBytes 11.5 Mbits/sec
[ 1] 11.0000-11.5000 sec 955 KBytes 15.7 Mbits/sec
[ 1] 11.5000-12.0000 sec 1019 KBytes 16.7 Mbits/sec
[ 1] 12.0000-12.5000 sec 1.06 MBytes 17.7 Mbits/sec
[ 1] 12.5000-13.0000 sec 1.18 MBytes 19.8 Mbits/sec
[ 1] 13.0000-13.5000 sec 1.24 MBytes 20.9 Mbits/sec
[ 1] 13.5000-14.0000 sec 1.37 MBytes 22.9 Mbits/sec
[ 1] 14.0000-14.5000 sec 1.37 MBytes 22.9 Mbits/sec
[ 1] 14.5000-15.0000 sec 1.30 MBytes 21.9 Mbits/sec
[ 1] 15.0000-15.5000 sec 60.7 KBytes 994 Kbits/sec
[ 1] 15.5000-16.0000 sec 0.000 Bytes 0.000 bits/sec
[ 1] 16.0000-16.5000 sec 829 KBytes 13.6 Mbits/sec
[ 1] 16.5000-17.0000 sec 1.68 MBytes 28.2 Mbits/sec
[ 1] 17.0000-17.5000 sec 375 KBytes 6.15 Mbits/sec
[ 1] 17.5000-18.0000 sec 127 KBytes 2.09 Mbits/sec
[ 1] 18.0000-18.5000 sec 511 KBytes 8.38 Mbits/sec
[ 1] 18.5000-19.0000 sec 1.30 MBytes 21.9 Mbits/sec
[ 1] 19.0000-19.5000 sec 508 KBytes 8.33 Mbits/sec
[ 1] 19.5000-20.0000 sec 573 KBytes 9.38 Mbits/sec
[ 1] 20.0000-25.6845 sec 127 KBytes 182 Kbits/sec
[ 1] 0.0000-25.6845 sec 33.1 MBytes 10.8 Mbits/sec

*** Stopping 0 controllers
*** Stopping 5 links
****
*** Stopping 2 switches
S1 S2
*** Stopping 4 hosts
H1 H2 H3 H4
*** Done
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$
```

现象：整体带宽较高且比较稳定，0-10s flow1平均吞吐量基本稳定在12-13 Mb/s 这一量级；10-20s，flow1的带宽开始震荡，从6-28不等（我认为最后的是异常值，大概是因为最后的一个包不到时间就发送完导致的），flow2也是在不断震荡，从最开始的25Mb/s到最后的12.5，和flow1竞争；20-30s flow2的带宽虽然有震荡，但是大体还是维持在十几二十的水平。

整体看flow1平平均9.23Mb/s，flow2平均10.28Mb/s，二者相加占满20Mb/s的瓶颈链路带宽。

丢包率为1%

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
===== Flow1 Result (H1 -> H3) =====
-----
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 58198 connected with 10.0.0.3 port 5001 (icwnd/mss/irrt=14/1448/24957)
[ ID] Interval          Transfer          Bandwidth
[ 1] 0.0000-0.5000 sec    896 KBytes       14.7 Mbits/sec
[ 1] 0.5000-1.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 1.0000-1.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 1.5000-2.0000 sec    308 KBytes       5.04 Mbits/sec
[ 1] 2.0000-2.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 2.5000-3.0000 sec    384 KBytes       6.29 Mbits/sec
[ 1] 3.0000-3.5000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 3.5000-4.0000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 4.0000-4.5000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 4.5000-5.0000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 5.0000-5.5000 sec    384 KBytes       6.29 Mbits/sec
[ 1] 5.5000-6.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 6.0000-6.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 6.5000-7.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 7.0000-7.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 7.5000-8.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 8.0000-8.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 8.5000-9.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 9.0000-9.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 9.5000-10.0000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 10.0000-10.5000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 10.5000-11.0000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 11.0000-11.5000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 11.5000-12.0000 sec   256 KBytes       4.19 Mbits/sec
[ 1] 12.0000-12.5000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 12.5000-13.0000 sec   384 KBytes       6.29 Mbits/sec

(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
[ 1] 5.0000-5.5000 sec    384 KBytes       6.29 Mbits/sec
[ 1] 5.5000-6.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 6.0000-6.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 6.5000-7.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 7.0000-7.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 7.5000-8.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 8.0000-8.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 8.5000-9.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 9.0000-9.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 9.5000-10.0000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 10.0000-10.5000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 10.5000-11.0000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 11.0000-11.5000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 11.5000-12.0000 sec   256 KBytes       4.19 Mbits/sec
[ 1] 12.0000-12.5000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 12.5000-13.0000 sec   384 KBytes       6.29 Mbits/sec
[ 1] 13.0000-13.5000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 13.5000-14.0000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 14.0000-14.5000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 14.5000-15.0000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 15.0000-15.5000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 15.5000-16.0000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 16.0000-16.5000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 16.5000-17.0000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 17.0000-17.5000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 17.5000-18.0000 sec   384 KBytes       6.29 Mbits/sec
[ 1] 18.0000-18.5000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 18.5000-19.0000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 19.0000-19.5000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 19.5000-20.0000 sec   640 KBytes       10.5 Mbits/sec
[ 1] 20.0000-20.4854 sec   128 KBytes       2.16 Mbits/sec
[ 1] 0.0000-20.4854 sec  21.8 MBytes      8.93 Mbits/sec
```

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
===== Flow2 Result (H2 -> H4) =====
-----
Client connecting to 10.0.0.4, TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 33742 connected with 10.0.0.4 port 5002 (icwnd/mss/irrt=14/1448/89354)
[ ID] Interval          Transfer          Bandwidth
[ 1] 0.0000-0.5000 sec    1.13 MBytes      18.9 Mbits/sec
[ 1] 0.5000-1.0000 sec    384 KBytes       6.29 Mbits/sec
[ 1] 1.0000-1.5000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 1.5000-2.0000 sec    377 KBytes       6.18 Mbits/sec
[ 1] 2.0000-2.5000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 2.5000-3.0000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 3.0000-3.5000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 3.5000-4.0000 sec    318 KBytes       5.21 Mbits/sec
[ 1] 4.0000-4.5000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 4.5000-5.0000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 5.0000-5.5000 sec    318 KBytes       5.21 Mbits/sec
[ 1] 5.5000-6.0000 sec    446 KBytes       7.31 Mbits/sec
[ 1] 6.0000-6.5000 sec    318 KBytes       5.21 Mbits/sec
[ 1] 6.5000-7.0000 sec    382 KBytes       6.26 Mbits/sec
[ 1] 7.0000-7.5000 sec    445 KBytes       7.30 Mbits/sec
[ 1] 7.5000-8.0000 sec    190 KBytes       3.12 Mbits/sec
[ 1] 8.0000-8.5000 sec    255 KBytes       4.18 Mbits/sec
[ 1] 8.5000-9.0000 sec    127 KBytes       2.07 Mbits/sec
[ 1] 9.0000-9.5000 sec    383 KBytes       6.27 Mbits/sec
[ 1] 9.5000-10.0000 sec   190 KBytes       3.12 Mbits/sec
[ 1] 10.0000-10.5000 sec   319 KBytes       5.22 Mbits/sec
[ 1] 10.5000-11.0000 sec   190 KBytes       3.12 Mbits/sec
[ 1] 11.0000-11.5000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 11.5000-12.0000 sec   383 KBytes       6.27 Mbits/sec
[ 1] 12.0000-12.5000 sec   254 KBytes       4.16 Mbits/sec
[ 1] 12.5000-13.0000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 10.0000-10.5000 sec   319 KBytes       5.22 Mbits/sec
[ 1] 10.5000-11.0000 sec   190 KBytes       3.12 Mbits/sec
[ 1] 11.0000-11.5000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 11.5000-12.0000 sec   383 KBytes       6.27 Mbits/sec
[ 1] 12.0000-12.5000 sec   254 KBytes       4.16 Mbits/sec
[ 1] 12.5000-13.0000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 13.0000-13.5000 sec   383 KBytes       6.27 Mbits/sec
[ 1] 13.5000-14.0000 sec   254 KBytes       4.16 Mbits/sec
[ 1] 14.0000-14.5000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 14.5000-15.0000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 15.0000-15.5000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 15.5000-16.0000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 16.0000-16.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 16.5000-17.0000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 17.0000-17.5000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 17.5000-18.0000 sec   383 KBytes       6.27 Mbits/sec
[ 1] 18.0000-18.5000 sec   127 KBytes       2.07 Mbits/sec
[ 1] 18.5000-19.0000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 19.0000-19.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 19.5000-20.0000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 20.0000-21.7545 sec   127 KBytes       594 Kbits/sec
[ 1] 0.0000-21.7545 sec  11.3 MBytes      4.36 Mbits/sec

*** Stopping 0 controllers
*** Stopping 5 links
.....
*** Stopping 2 switches
S1 S2
*** Stopping 4 hosts
H1 H2 H3 H4
*** Done
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$
```

现象：0-10s flow1的带宽明显下降，基本在6-10，而且震荡剧烈（因为有重传）；10-20s flow1 维持了震荡态势，flow2 的带宽则更小，在3-7左右；20-30s flow2的带宽也没有回升很多，只是稍微增加。

整体看flow1平均8.93Mb/s flow2平均4.36Mb/s（看来这个链路的公平性也不太好），当然和下面更极端的丢包率相比现在还没有出现0Mb/s

丢包率为5%

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
===== Flow1 Result (H1 -> H3) =====
-----
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 39422 connected with 10.0.0.3 port 5001 (icwnd/mss/irrt=14/1448/56054)
[ ID] Interval          Transfer          Bandwidth
[ 1] 0.0000-0.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 0.5000-1.0000 sec    207 KBytes       3.39 Mbits/sec
[ 1] 1.0000-1.5000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 1.5000-2.0000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 2.0000-2.5000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 2.5000-3.0000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 3.0000-3.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 3.5000-4.0000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 4.0000-4.5000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 4.5000-5.0000 sec    255 KBytes       4.18 Mbits/sec
[ 1] 5.0000-5.5000 sec    254 KBytes       4.16 Mbits/sec
[ 1] 5.5000-6.0000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 6.0000-6.5000 sec    384 KBytes       6.29 Mbits/sec
[ 1] 6.5000-7.0000 sec    252 KBytes       4.13 Mbits/sec
[ 1] 7.0000-7.5000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 7.5000-8.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 8.0000-8.5000 sec    316 KBytes       5.18 Mbits/sec
[ 1] 8.5000-9.0000 sec    256 KBytes       4.19 Mbits/sec
[ 1] 9.0000-9.5000 sec    384 KBytes       6.29 Mbits/sec
[ 1] 9.5000-10.0000 sec   251 KBytes       4.11 Mbits/sec
[ 1] 10.0000-10.5000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 10.5000-11.0000 sec   254 KBytes       4.16 Mbits/sec
[ 1] 11.0000-11.5000 sec   256 KBytes       4.19 Mbits/sec
[ 1] 11.5000-12.0000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 12.0000-12.5000 sec   250 KBytes       4.10 Mbits/sec
[ 1] 12.5000-13.0000 sec   192 KBytes       3.14 Mbits/sec
```

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
[ 1] 5.5000-6.0000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 6.0000-6.5000 sec    384 KBytes       6.29 Mbits/sec
[ 1] 6.5000-7.0000 sec    252 KBytes       4.13 Mbits/sec
[ 1] 7.0000-7.5000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 7.5000-8.0000 sec    512 KBytes       8.39 Mbits/sec
[ 1] 8.0000-8.5000 sec    316 KBytes       5.18 Mbits/sec
[ 1] 8.5000-9.0000 sec    256 KBytes       4.19 Mbits/sec
[ 1] 9.0000-9.5000 sec    384 KBytes       6.29 Mbits/sec
[ 1] 9.5000-10.0000 sec   251 KBytes       4.11 Mbits/sec
[ 1] 10.0000-10.5000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 10.5000-11.0000 sec   254 KBytes       4.16 Mbits/sec
[ 1] 11.0000-11.5000 sec   256 KBytes       4.19 Mbits/sec
[ 1] 11.5000-12.0000 sec   512 KBytes       8.39 Mbits/sec
[ 1] 12.0000-12.5000 sec   250 KBytes       4.10 Mbits/sec
[ 1] 12.5000-13.0000 sec   192 KBytes       3.14 Mbits/sec
[ 1] 13.0000-13.5000 sec   384 KBytes       6.29 Mbits/sec
[ 1] 13.5000-14.0000 sec   188 KBytes       3.08 Mbits/sec
[ 1] 14.0000-14.5000 sec   192 KBytes       3.14 Mbits/sec
[ 1] 14.5000-15.0000 sec   254 KBytes       4.16 Mbits/sec
[ 1] 15.0000-15.5000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 15.5000-16.0000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 16.0000-16.5000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 16.5000-17.0000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 17.0000-17.5000 sec   383 KBytes       6.27 Mbits/sec
[ 1] 17.5000-18.0000 sec   256 KBytes       4.19 Mbits/sec
[ 1] 18.0000-18.5000 sec   189 KBytes       3.09 Mbits/sec
[ 1] 18.5000-19.0000 sec   191 KBytes       3.13 Mbits/sec
[ 1] 19.0000-19.5000 sec   192 KBytes       3.14 Mbits/sec
[ 1] 19.5000-20.0000 sec   319 KBytes       5.22 Mbits/sec
[ 1] 20.0000-21.4407 sec   128 KBytes       728 Kbits/sec
[ 1] 0.0000-21.4407 sec  10.6 MBytes      4.14 Mbits/sec
```

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py

===== Flow2 Result (H2 -> H4) =====
-----
Client connecting to 10.0.0.4, TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 49534 connected with 10.0.0.4 port 5002 (icwnd/mss/irrt=14/1448/89648)
[ ID] Interval          Transfer          Bandwidth
[ 1] 0.0000-0.5000 sec    255 KBytes       4.17 Mbits/sec
[ 1] 0.5000-1.0000 sec    161 KBytes       2.64 Mbits/sec
[ 1] 1.0000-1.5000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 1.5000-2.0000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 2.0000-2.5000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 2.5000-3.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 3.0000-3.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 3.5000-4.0000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 4.0000-4.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 4.5000-5.0000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 5.0000-5.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 5.5000-6.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 6.0000-6.5000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 6.5000-7.0000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 7.0000-7.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 7.5000-8.0000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 8.0000-8.5000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 8.5000-9.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 9.0000-9.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 9.5000-10.0000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 10.0000-10.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 10.5000-11.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 11.0000-11.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 11.5000-12.0000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 12.0000-12.5000 sec   0.000 Bytes      0.000 bits/sec

(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
[ 1] 10.0000-10.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 10.5000-11.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 11.0000-11.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 11.5000-12.0000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 12.0000-12.5000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 12.5000-13.0000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 13.0000-13.5000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 13.5000-14.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 14.0000-14.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 14.5000-15.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 15.0000-15.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 15.5000-16.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 16.0000-16.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 16.5000-17.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 17.0000-17.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 17.5000-18.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 18.0000-18.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 18.5000-19.0000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 19.0000-19.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 19.5000-20.0000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 20.0000-21.3314 sec   63.6 KBytes      392 Kbits/sec
[ 1] 0.0000-21.3314 sec   3.45 MBytes      1.36 Mbits/sec

*** Stopping 0 controllers

*** Stopping 5 links
.....
*** Stopping 2 switches
S1 S2
*** Stopping 4 hosts
H1 H2 H3 H4
*** Done
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$
```

现象： 震荡的幅度变小了，0-10s flow1基本稳定在4多一点；10-20s flow1 则几乎维持在3-4左右，flow2 则是逐渐下降，只有1Mb/s；20-30s flow2的情况也没有好转，而且还有很多重传和超时导致的0Mb/s的部分

整体看flow1平均4.14Mb/s flow2平均1.36Mb/s

丢包率为10%

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
===== Flow1 Result (H1 -> H3) =====
-----
Client connecting to 10.0.0.3, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 35264 connected with 10.0.0.3 port 5001 (icwnd/mss/irrt=14/1448/1055000)
[ ID] Interval          Transfer          Bandwidth
[ 1] 0.0000-0.5000 sec    640 KBytes       10.5 Mbits/sec
[ 1] 0.5000-1.0000 sec    139 KBytes       2.28 Mbits/sec
[ 1] 1.0000-1.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 1.5000-2.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 2.0000-2.5000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 2.5000-3.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 3.0000-3.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 3.5000-4.0000 sec    256 KBytes       4.19 Mbits/sec
[ 1] 4.0000-4.5000 sec    126 KBytes       2.06 Mbits/sec
[ 1] 4.5000-5.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 5.0000-5.5000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 5.5000-6.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 6.0000-6.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 6.5000-7.0000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 7.0000-7.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 7.5000-8.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 8.0000-8.5000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 8.5000-9.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 9.0000-9.5000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 9.5000-10.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 10.0000-10.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 10.5000-11.0000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 11.0000-11.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 11.5000-12.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 12.0000-12.5000 sec   127 KBytes       2.09 Mbits/sec

(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
[ 1] 5.5000-6.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 6.0000-6.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 6.5000-7.0000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 7.0000-7.5000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 7.5000-8.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 8.0000-8.5000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 8.5000-9.0000 sec    63.6 KBytes      1.04 Mbits/sec
[ 1] 9.0000-9.5000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 9.5000-10.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 10.0000-10.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 10.5000-11.0000 sec   255 KBytes       4.17 Mbits/sec
[ 1] 11.0000-11.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 11.5000-12.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 12.0000-12.5000 sec   127 KBytes       2.09 Mbits/sec
[ 1] 12.5000-13.0000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 13.0000-13.5000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 13.5000-14.0000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 14.0000-14.5000 sec    0.000 Bytes      0.000 bits/sec
[ 1] 14.5000-15.0000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 15.0000-15.5000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 15.5000-16.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 16.0000-16.5000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 16.5000-17.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 17.0000-17.5000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 17.5000-18.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 18.0000-18.5000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 18.5000-19.0000 sec   63.6 KBytes      1.04 Mbits/sec
[ 1] 19.0000-19.5000 sec    191 KBytes       3.13 Mbits/sec
[ 1] 19.5000-20.0000 sec    127 KBytes       2.09 Mbits/sec
[ 1] 20.0000-21.5063 sec   63.6 KBytes      346 Kbits/sec
[ 1] 0.0000-21.5063 sec   4.24 MBytes      1.65 Mbits/sec
```

```
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
===== Flow2 Result (H2 -> H4) =====
-----
Client connecting to 10.0.0.4, TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 44186 connected with 10.0.0.4 port 5002 (icwnd/mss/irrt=14/1448/89655)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-0.5000 sec  190 KBytes    3.11 Mbits/sec
[ 1] 0.5000-1.0000 sec  97.6 KBytes    1.60 Mbits/sec
[ 1] 1.0000-1.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 1.5000-2.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 2.0000-2.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 2.5000-3.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 3.0000-3.5000 sec  127 KBytes    2.09 Mbits/sec
[ 1] 3.5000-4.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 4.0000-4.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 4.5000-5.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 5.0000-5.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 5.5000-6.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 6.0000-6.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 6.5000-7.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 7.0000-7.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 7.5000-8.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 8.0000-8.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 8.5000-9.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 9.0000-9.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 9.5000-10.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 10.0000-10.5000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 10.5000-11.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 11.0000-11.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 11.5000-12.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 12.0000-12.5000 sec   0.000 Bytes    0.000 bits/sec

(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$ sudo python3 host iperf.py
[ 1] 10.0000-10.5000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 10.5000-11.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 11.0000-11.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 11.5000-12.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 12.0000-12.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 12.5000-13.0000 sec  127 KBytes    2.09 Mbits/sec
[ 1] 13.0000-13.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 13.5000-14.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 14.0000-14.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 14.5000-15.0000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 15.0000-15.5000 sec  63.6 KBytes    1.04 Mbits/sec
[ 1] 15.5000-16.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 16.0000-16.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 16.5000-17.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 17.0000-17.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 17.5000-18.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 18.0000-18.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 18.5000-19.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 19.0000-19.5000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 19.5000-20.0000 sec   0.000 Bytes    0.000 bits/sec
[ 1] 0.0000-27.3879 sec  1.27 MBytes    390 Kbits/sec

*** Stopping 0 controllers

*** Stopping 5 links
.....
*** Stopping 2 switches
S1 S2
*** Stopping 4 hosts
H1 H2 H3 H4
*** Done
(base) vagrant@vagrant-VMware-Virtual-Platform:~/Documents/computerNetworkLab5$
```

现象：0-10s flow1几乎维持1.0几的吞吐量；10-20s 除了继续维持1.0几外还有一段全0的区间，flow2 则是0和1断断续续；20-30s flow2几乎都是0Mb/s

整体看flow1平均1.65Mb/s flow2平均0.39Mb/s

现象分析

首先，可以看到平均吞吐量随丢包率上升而下降 $\text{Throughput}_{\text{TCP}} = \frac{\text{MSS}}{\text{RTT} \times \sqrt{p}}$ 可以说是很符合这个公式的，丢包率高时cwnd一直很小

另外，而且明显看到flow2竞争不过flow1。理来说TCP应该能收敛到公平，但是flow2从H2 s1 s2到H4一路上延迟大约22ms，远高于flow1的6ms，因此RTT更大，使得丢包率低时二者还是公平的；丢包率高时则flow2的吞吐量远低于flow1（当然也符合上述公式）

参考

<https://github.com/CPqD/ofsoftswitch13/issues/108> <https://stackoverflow.com/questions/63907768/use-iperf-and-ping-at-the-same-time-mininet> <https://tyler-ytr.github.io/2021/09/03/mininetlab/>
<https://www.runoob.com/linux/linux-comm-iperf.html> <https://mininet-optical.org/api-overview.html>